

Caso 2 — Línea de Producción con Balanceo Dinámico

Documento de trabajo: la idea en lenguaje cotidiano, los conceptos del curso, la arquitectura propuesta y el plan de ejecución.

Entrega: lunes 24/08/2026, 11:30 hrs. — informe PDF, presentación PDF, código fuente (.zip) y demo funcional.

1. Qué hay que construir

La evaluación pide diseñar, implementar y **desplegar** un sistema distribuido que simule una línea de ensamble industrial. El problema que da sentido al caso es que el **cuello de botella se desplaza**: la estación que hoy frena la línea puede no ser la misma en diez minutos. Frente a eso, el sistema debe replicar una estación en varias instancias, repartir las órdenes entre ellas sin que dos tomen la misma, y detectar en vivo dónde se está acumulando el trabajo.

Exigencia del enunciado	Cómo la cubrimos
Mínimo 3 servicios o procesos	Cuatro: Órdenes, Estación (replicada), Coordinación de carga y Métricas
Mínimo 2 nodos lógicos	Tres contenedores Docker, uno por grupo funcional
UI con interacción real	Streamlit: crea órdenes, ajusta tiempos y muestra el estado en vivo
Persistencia obligatoria	Base de datos con órdenes, eventos e historial de métricas
Reproducibilidad	Todo el sistema levanta con docker compose up , más README
Manejo básico de errores	Validación de entradas, códigos HTTP correctos y mensajes claros en la UI

2. La idea completa, sin un solo término técnico

Antes de cualquier definición conviene tener una imagen mental concreta. Lo que vamos a construir es, punto por punto, la versión en software de la cocina de una sandwichería en hora punta. Si se entiende la cocina, se entiende el trabajo completo.

2.1. La cocina

Llega un cliente al mesón y pide un italiano. El cajero anota el pedido en un papelito y lo cuelga en un riel. Detrás hay tres sandwicheros trabajando en paralelo: cada vez que uno termina un sándwich, se da vuelta, saca el siguiente papelito del riel y lo prepara. En la pared hay una pizarra donde el dueño va anotando cuánto se demoró cada pedido.

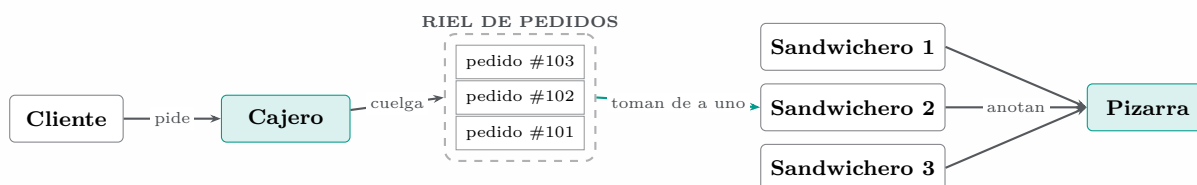


Figura 1: La cocina en hora punta. Todo el trabajo consiste en construir esto en software.

2.2. Qué sale mal, y por qué existe este trabajo

Dos sandwicheros terminan al mismo tiempo. Los dos se dan vuelta, los dos estiran la mano y los dos agarran *el mismo* papelito. Resultado: se preparan dos italianos idénticos, uno se bota, y el

cliente que venía después sigue esperando sin que nadie se haya dado cuenta.

En una cocina de verdad esto casi no pasa, porque los tres se ven las caras y uno le grita al otro. Pero imaginen ahora que cada sandwichero está en una pieza separada, sin verse ni oírse, y que cada uno mira una copia del riel por una pantalla. Ahí el choque pasaría todo el rato. **Esa cocina de piezas separadas es exactamente un sistema distribuido:** los que trabajan no comparten el mismo espacio, no se ven, y por lo tanto hay que inventar una regla explícita para que no se pisen.

2.3. Cómo se arregla

La solución de la cocina es poner a alguien en el medio. Un *cantador* se queda con el riel y va entregando los papelitos de a uno: nadie toma nada por su cuenta, todos le piden a él. Como el cantador atiende a una persona a la vez, es imposible que dos reciban el mismo pedido.

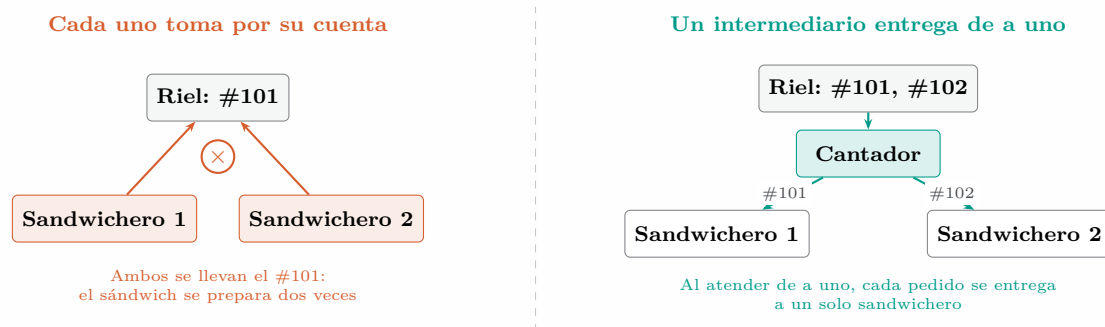


Figura 2: El choque entre dos trabajadores y su solución mediante un intermediario que entrega de a uno.

Y aquí aparece algo que vale la pena notar, porque es el título del caso. Como cada sandwichero pide el siguiente papelito *recién cuando se desocupa*, el que trabaja más rápido termina haciendo más sándwiches, sin que nadie haya tenido que repartir el trabajo por adelantado. Si uno anda lento porque le tocó un pedido complicado, los otros dos simplemente piden más seguido y absorben la diferencia. **El trabajo se reparte solo.**

2.4. Cómo saber dónde se está atascando el local

Si la fila de papelitos en el riel crece y crece, algo anda mal. Pero cuidado con la conclusión fácil. Que armar un italiano tome cuatro minutos no es un problema: eso es simplemente lo que demora un italiano. El problema es que un papelito lleve veinte minutos colgado antes de que alguien lo tome siquiera.

Lo primero es el *trabajo*; lo segundo es el *atasco*. Y solo lo segundo indica que hace falta otro sandwichero. Distinguir esos dos tiempos es todo el criterio de diagnóstico del sistema, y es lo que hay que tener claro antes de escribir una línea de código.

2.5. Diccionario: de la cocina a la materia

En la cocina	Nombre en el curso	Qué es exactamente
Sandwicheros en piezas separadas	Sistema distribuido	Componentes autónomos que no comparten memoria y coordinan enviándose mensajes por la red
El papelito del pedido	Orden / mensaje	Estructura de datos en formato JSON que viaja de un servicio a otro
El riel	Cola de trabajo	Estructura donde se acumulan las órdenes pendientes de ser tomadas
Cada sandwichero	Réplica de la estación	Instancia idéntica del mismo servicio, ejecutándose en paralelo
Que dos tomen el mismo papelito	Condición de carrera	Resultado que depende del orden imprevisible en que ocurren dos accesos simultáneos
El cantador	Exclusión mutua distribuida	Mecanismo que garantiza que solo un proceso accede a un recurso compartido a la vez
Que el más rápido haga más	Balanceo dinámico de carga	Reparto por demanda del trabajador, en vez de asignación previa y fija
Lo que espera el cliente	Lead time	Tiempo total desde que la orden entra hasta que se completa
El riel que crece	Cuello de botella	Etapas cuya capacidad limita el rendimiento del sistema completo
La pizarra del dueño	Servicio de métricas	Servicio que calcula tiempos e identifica dónde se acumula el trabajo
El mesón del cajero	Interfaz de usuario	Aplicación web desde la que se opera el sistema

3. Los mismos conceptos, ahora en el lenguaje del curso

3.1. Sistema distribuido, nodo y servicio

En la cocina: *tres sandwicheros en piezas separadas que no se ven ni se oyen.*

Un sistema distribuido es una colección de computadores independientes que se presentan al usuario como un único sistema coherente. La diferencia esencial con un programa normal es que **no hay memoria compartida ni un reloj común**: cada parte solo sabe lo que le han contado por la red, y siempre con retraso.

Un **nodo lógico** es cada entorno de ejecución aislado; en nuestro caso, cada contenedor Docker. Un **servicio** es un programa con una sola responsabilidad que expone una interfaz para que otros lo usen. Aquí esa interfaz será REST sobre HTTP: un servicio le hace una petición a otro por la red y recibe una respuesta, intercambiando mensajes en formato JSON.

3.2. Condición de carrera y exclusión mutua distribuida

En la cocina: *los dos sandwicheros estirando la mano hacia el mismo papelito.*

Cuando dos procesos acceden al mismo recurso al mismo tiempo y el resultado depende de cuál llegó primero, se produce una **condición de carrera**. En un solo computador el problema se resuelve con un *mutex*: una marca en memoria que los hilos revisan antes de entrar a la zona conflictiva. Pero en un sistema distribuido no existe esa memoria común, así que se necesita un árbitro externo al que todos le pregunten. Eso es la **exclusión mutua distribuida**.

Nuestro árbitro será **Redis**, que ofrece operaciones atómicas sobre una cola. Atómica significa que la operación ocurre completa o no ocurre: no puede quedar a medias ni entrelazarse con otra. Aunque dos réplicas pidan en el mismo milisegundo, la operación entrega la orden a una sola y la otra recibe la siguiente. Es la Figura 2, cambiando *sandwichero* por *réplica* y *cantador* por *Redis*.

3.3. Balanceo dinámico de carga

En la cocina: *el que se desocupa primero pide el siguiente pedido.*

Repartir el trabajo de forma **estática** — por ejemplo, las órdenes pares a la réplica 1 y las impares a la réplica 2 — funciona solo mientras todas rindan igual. Si una se degrada, su mitad de las órdenes se acumula aunque las demás estén ociosas. El **balanceo dinámico** invierte la lógica: nadie asigna trabajo por adelantado, sino que cada réplica pide la siguiente orden cuando queda libre. La cola compartida nos entrega ese comportamiento sin lógica adicional.

3.4. Cómo se mide el rendimiento de la línea

En la cocina: *lo que el cliente espera, contra lo que demora armar el sándwich.*

Toda orden atraviesa dos etapas: primero *espera* en la cola, y después es *procesada*. Separar ambos tiempos es lo que permite localizar el cuello de botella.

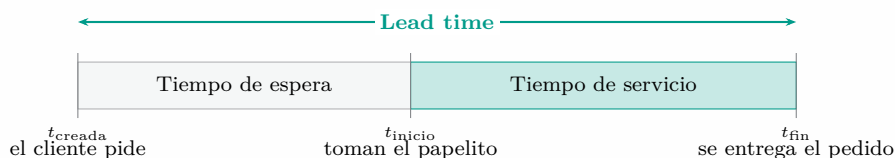


Figura 3: Ciclo de vida de una orden. El lead time es la suma de la espera y el servicio.

$$\text{Lead time} = t_{\text{fin}} - t_{\text{creada}} \quad \text{Utilización}_i = \frac{\text{tiempo ocupado de la réplica } i}{\text{tiempo total observado}}$$

La clave de diagnóstico es que **un tiempo de servicio alto no es un cuello de botella**: significa simplemente que la tarea es larga. El cuello de botella aparece cuando el *tiempo de espera* crece de forma sostenida, porque indica que llega más trabajo del que la estación puede absorber. Sobre esa señal definiremos los tres estados que pide el enunciado: **normal**, **advertencia** y **crítico**.

4. Arquitectura propuesta

Los cuatro servicios se reparten en tres nodos lógicos, y esa división no es arbitraria: agrupa responsabilidades que cambian de escala por motivos distintos. El nodo B es el único que necesita replicarse cuando sube la demanda; el nodo C concentra el estado compartido, que precisamente por ser compartido no puede duplicarse sin coordinación; y el nodo A expone el sistema hacia el operador. Esa separación es la que justifica la arquitectura distribuida frente a una aplicación monolítica, y es el argumento que hay que sostener en el informe y en la defensa.

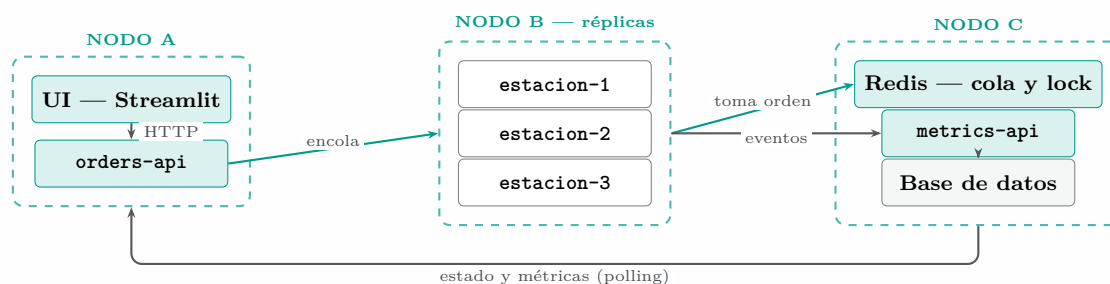


Figura 4: La misma cocina, con los nombres del curso: cajero y mesón en el nodo A, sandwicheros en el B, riel, cantador y pizarra en el C.



Servicio	Responsabilidad	Tecnología
Órdenes	Recibe, valida y encola las órdenes de producción	FastAPI
Estación	Procesa órdenes con tiempos configurables y clasifica su estado	FastAPI (2-3 réplicas)
Coordinación de carga	Entrega cada orden a una sola réplica mediante operaciones atómicas	Redis
Métricas	Calcula lead time por réplica y detecta el cuello de botella	FastAPI
Interfaz de usuario	Crea órdenes, ajusta parámetros y visualiza el estado de la línea	Streamlit

5. Plan de ejecución

Día	Objetivo
Dom 16	Instalar Docker, crear el repositorio Git y confirmar con el profesor el caso asignado y la conformación del grupo
Lun 17	Definir en conjunto los contratos de las API, el esquema de datos y los umbrales de las métricas
Mar 18	Órdenes, estación y cola con Redis. <i>Meta: una orden entra por la API y sale procesada</i>
Mié 19	Métricas y UI. Punto de control sobre el alcance final
Jue 20	Integración, validaciones, logging, <code>docker-compose</code> y README
Vie 21	Experimentos con 1, 2 y 3 réplicas; provocar un cuello de botella y registrar los resultados
Sáb 22	Redacción del informe
Dom 23	Presentación y ensayo del demo

El informe se escribe **en paralelo desde el lunes**, no al final: el marco teórico no depende del código y concentra buena parte del puntaje. Los commits deben repartirse a lo largo de la semana, ya que la gestión mediante repositorio Git se evalúa de forma explícita.

5.1. Decisiones que quedan abiertas

- Criterio de cuello de botella:** definir si se mide por largo de cola, por utilización de la réplica o por tiempo de espera promedio, y fijar los umbrales de advertencia y crítico.
- Mecanismo de coordinación:** extracción atómica desde la cola de Redis frente a un bloqueo explícito con tiempo de expiración. La primera opción es más simple y más difícil de romper.
- Alcance ampliado:** evaluar el miércoles la incorporación de un *broker* de mensajería para el transporte de eventos entre servicios, condicionada a que el núcleo esté operativo.